AI Software Incident Response Agent

5-Hour Hackathon Plan: architecture, tech stack, setup, execution plan and agent prompts

Note for Codex: This document is the full project specification. Read it end to end, then follow Section 9 (*Prompts to give your coding agents*) and Section 5 (*Hour-by-hour plan*). Build the backend in `/backend` and keep the API contract in `/docs/API.md`.

0. Problem statement (PS-61)

Field	Value
Title	PS-61: AI Software Incident Response Agent
Target domain	Agentic AI, Web Development, Vibe Coding
Difficulty	Advanced
Industry	Cybersecurity / Software Engineering / Enterprise Security
Scope	Software Incident Detection / Incident Response / Root Cause Analysis / Security Operations
Platform / tech	Agentic AI, AI/ML, NLP, Log Analysis, Anomaly Detection, Web Application, Workflow Automation, Security Analytics

Description. Develop an AI-powered software incident response agent that assists development and security teams in detecting, investigating, and responding to software incidents. The system should analyze incident information such as application logs, alerts, error reports, system events, and historical incidents to identify potential causes, correlate related events, recommend response actions, and track incidents through resolution. The agent should support human review and provide explanations for its findings and recommendations.

Key deliverables

- Incident creation and intake
- Integration with application/system logs and alerts
- Automated incident classification
- Severity and priority assessment
- Event and log correlation
- Anomaly and error detection
- Root-cause analysis assistance
- Similar/historical incident identification
- AI-generated incident summaries
- Recommended investigation and remediation steps
- Automated incident-response workflow support
- Assignment and escalation management
- Incident status and lifecycle tracking
- Human approval before sensitive response actions
- Evidence and investigation timeline
- Notifications and escalation alerts
- Incident dashboard
- Search and filtering
- Historical incident knowledge base
- Resolution and post-incident documentation
- Incident analytics and reports
- Audit trail of AI recommendations and human actions

1. What to build

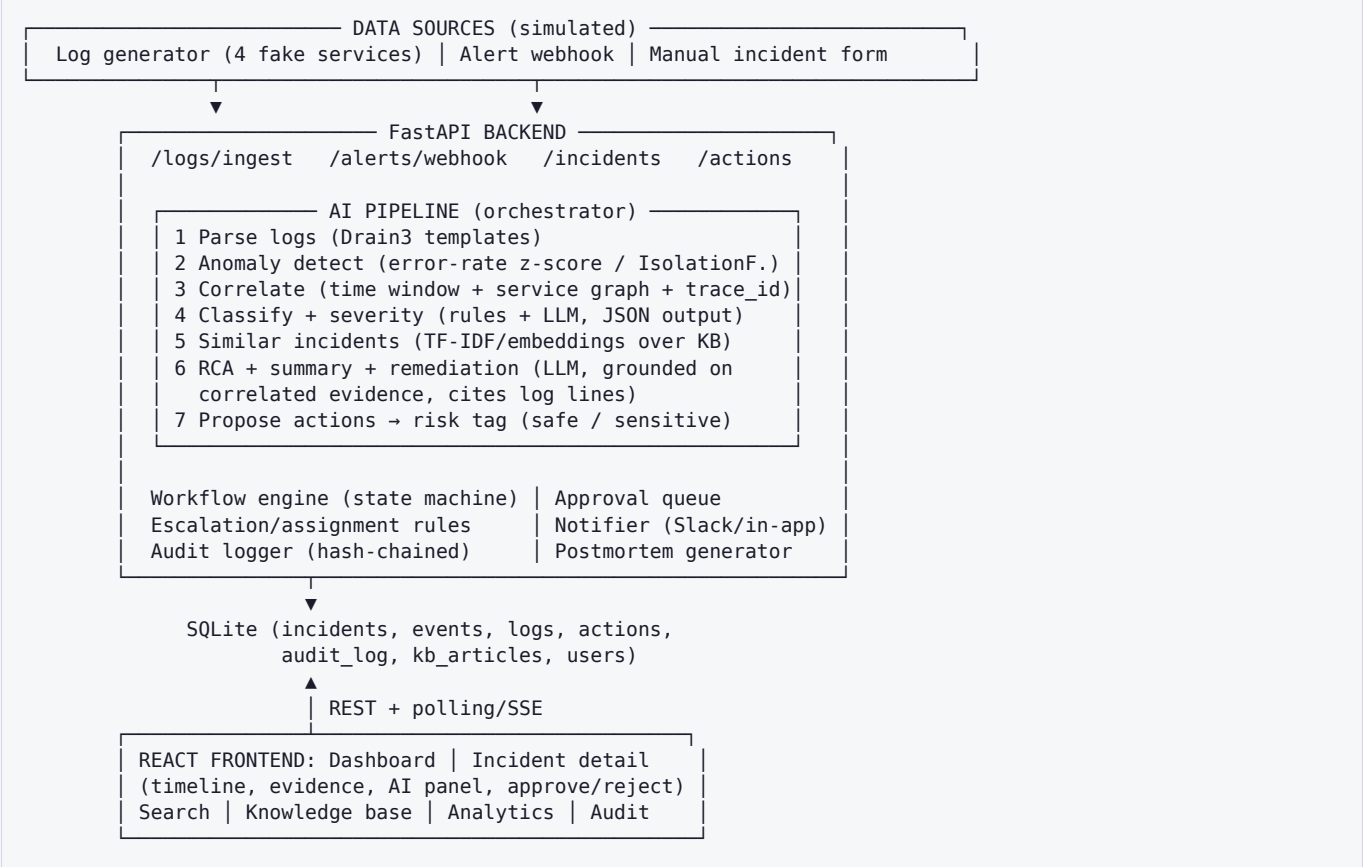
Don't try to build all 22 deliverables. Judges reward **one polished end-to-end story** over a wide, shallow build. Build one scenario that runs from alert to postmortem:

> **The demo story**
>
> Alert fires → incident auto-created → AI classifies, scores severity, correlates logs → finds a similar past incident → proposes root cause and fix → human approves a rollback → incident resolves → AI drafts the postmortem. Everything is logged in an audit trail.

Priority tiers

Tier	Features
Must (hours 0–3.5)	Incident intake (form + webhook), log ingestion, classification, severity/priority, correlation, anomaly detection, similar-incident search, AI summary + RCA + recommended steps, human approval gate, lifecycle states, timeline, audit trail, dashboard
Should (3.5–4.3)	Assignment/escalation rules, notifications (Slack webhook or in-app), search/filter, knowledge base, postmortem generator, analytics (MTTR, by severity/category)
Skip	Real integrations (Datadog, Splunk), auth, multi-tenancy, Docker/Kubernetes. Use mock log generators instead

2. Architecture



Design principles to state in your pitch

- **Human-in-the-loop:** the AI *proposes*, humans *approve*. Actions are tagged **safe** (auto-run, e.g. notify, gather logs) or **sensitive** (rollback, restart, block IP, which need approval).
- **Explainability:** every finding links to evidence (log lines, event IDs) with a confidence score.
- **Graceful fallback:** if the LLM call fails, rule-based results still render. This protects your live demo.

- **Audit trail:** every AI recommendation and human action is stored with timestamp and actor, hash-chained so tampering is detectable.

3. Tech stack

Layer	Choice	Why
Backend	**Python 3.11 + FastAPI + SQLModel + SQLite**	Fastest to build, zero DB setup
AI orchestration	Plain Python pipeline + Pydantic structured output	Fewer failure points than LangGraph in 5 hours. Mention LangGraph as future work
LLM	Any API you already have a key for (OpenAI, Gemini, Claude), with `LLM_PROVIDER` in `.env`	Keep it swappable. Ollama is an offline backup
Log parsing	**drain3**	Standard log-template mining
Anomaly detection	**scikit-learn** IsolationForest + rolling z-score	Simple, explainable
Similarity	**TF-IDF + cosine** (scikit-learn). Upgrade to sentence-transformers (all-MiniLM-L6-v2) only if time allows	TF-IDF needs no model download
Frontend	**React + Vite + Tailwind + shadcn/ui + Recharts**	Looks polished quickly. If short on time, **Streamlit** is the emergency fallback
Notifications	Slack Incoming Webhook + in-app toast feed	Webhook takes 5 minutes to set up

4. What to download and set up

Install

- Python 3.11+, Node 20+, Git
- Codex CLI and Antigravity (you already have these)

Python packages

```
pip install fastapi uvicorn sqlmodel pydantic python-dotenv httpx \
  scikit-learn pandas numpy drain3 apscheduler faker
# plus your LLM SDK: openai | google-genai | anthropic
```

Frontend

```
npm create vite@latest frontend -- --template react-ts
cd frontend && npm i tailwindcss recharts lucide-react axios react-router-dom
npx shadcn@latest init
```

Datasets (optional, for realism)

- **Loghub** (github.com/logpai/loghub): download small samples such as Apache, OpenStack, HDFS. Use them only as background noise.
- For the demo itself, **write your own synthetic generator** for 3–4 services (`api-gateway`, `checkout-service`, `payment-service`, `postgres`) with injectable failures. You control the story and the timing.

Seed content to prepare

- 15–20 historical incidents (JSON) with root cause, fix, and category. You can have the LLM generate these.
- 5–6 runbook/KB articles in Markdown.

5. Hour-by-hour plan

Time	Goal	Output
0:00–0:30	Setup and spec	Repo, `AGENTS.md`, DB schema, `.env`, seed data script
0:30–1:30	Backend core	Models, CRUD, log ingestion, state machine (New → Triaged → Investigating → Awaiting Approval → Mitigated → Resolved → Closed), audit logger
1:30–2:30	AI pipeline	Parser, anomaly detector, correlator, classifier/severity, similarity search, LLM RCA/summary/recommendations
1:30–3:30	Frontend (parallel)	Dashboard, incident detail page with timeline and AI panel, approval buttons
3:30–4:15	Extras	Escalation rules, Slack notify, KB page, analytics, postmortem generator
4:15–4:45	Demo hardening	Seed demo data, fix bugs, scripted "inject failure" button
4:45–5:00	Submission	README, architecture diagram, 3-minute demo rehearsal

Tool split: run **Codex on the backend** and **Antigravity on the frontend** in parallel in separate folders (`/backend`, `/frontend`). Agree on the API contract first so neither blocks the other. Write the OpenAPI shape in the spec before hour 0:30.

6. Repo structure

```
incident-agent/
├── AGENTS.md              # instructions for Codex/Antigravity
├── backend/
│   ├── main.py
│   ├── models.py          # Incident, Event, LogEntry, Action, AuditLog, KB
│   ├── routers/           # incidents, logs, actions, analytics
│   ├── pipeline/          # parser, anomaly, correlate, classify, similar, rca
│   ├── workflow.py        # state machine + escalation rules
│   ├── audit.py           # hash-chained audit log
│   ├── notify.py
│   ├── llm.py             # provider-agnostic wrapper + fallback
│   └── simulator.py       # fake logs + failure injection
├── data/                  # seed incidents, KB markdown
└── frontend/src/pages/    # Dashboard, IncidentDetail, KB, Analytics, Audit
```

7. Data model and API contract (starting point)

The tables and endpoints below follow directly from the plan above. Treat them as a starting contract and keep `/docs/API.md` in sync so the frontend and backend can be built in parallel.

Tables

Table	Key fields
`incidents`	id, title, description, source, category, severity (SEV1–SEV4), priority, status, assignee, confidence, ai_summary, probable_root_cause, created_at, resolved_at
`log_entries`	id, timestamp, service, level, message, template_id, trace_id, incident_id (nullable)
`timeline_events`	id, incident_id, timestamp, actor (system / ai / user), type, detail
`proposed_actions`	id, incident_id, description, risk (safe | sensitive), status (pending | approved | rejected | executed), decided_by, decided_at
`audit_log`	id, timestamp, actor, action, payload, prev_hash, hash
`kb_articles`	id, title, category, content_md, source_incident_id

Lifecycle states

New → Triaged → Investigating → Awaiting Approval → Mitigated → Resolved → Closed. Transitions are validated by the state machine, and every transition writes to `audit_log`.

Endpoints

Method / path	Purpose
`POST /incidents`	Create incident (manual form); triggers AI pipeline
`GET /incidents`	List with search and filters (status, severity, category, service, text)
`GET /incidents/{id}`	Detail with timeline, evidence, AI analysis, similar incidents, actions
`PATCH /incidents/{id}`	Update status, assignee; validated by state machine
`POST /logs/ingest`	Ingest log batches from the simulator
`POST /alerts/webhook`	Receive alert payload; auto-create incident
`POST /actions/{id}/approve`	Human approves a sensitive action
`POST /actions/{id}/reject`	Human rejects an action
`POST /incidents/{id}/postmortem`	Generate AI-drafted postmortem
`GET /kb`	Knowledge base articles and search
`GET /analytics`	MTTR, counts by severity and category
`GET /audit`	Audit trail with hash-chain verification
`POST /simulate/{scenario}`	Demo: inject a failure scenario

8. AGENTS.md (starter content)

Save this as `AGENTS.md` in the repo root so Codex and Antigravity follow the same rules.

```
# AGENTS.md

## Project
AI Software Incident Response Agent (PS-61). 5-hour hackathon build.

## Rules
- Backend: Python 3.11, FastAPI, SQLModel, SQLite. Frontend: React + Vite + Tailwind + shadcn/ui.
- Keep modules small. No auth, no Docker, no real third-party integrations.
- The LLM NEVER executes actions. It only proposes. Sensitive actions require human approval.
- All LLM output must be Pydantic-validated JSON. If the LLM fails, return the rule-based fallback.
- Every AI output and every state change writes to audit_log (hash-chained).
- Run the code and fix errors before stopping. Add a seed script and a simulator.
- Keep /docs/API.md in sync with the endpoints.

## Demo scenario
Bad deploy of checkout-service (v2.4.1) -> DB connection pool exhausted -> 5xx spike ->
incident SEV-1 -> AI finds similar past incident -> proposes Rollback to v2.4.0 (sensitive)
-> human approves -> Mitigated -> Resolved -> AI drafts postmortem.
```

9. Prompts to give your coding agents

First prompt (Codex, backend)

Build a FastAPI + SQLModel + SQLite backend for an AI incident response agent. Tables: incidents, log_entries, timeline_events, proposed_actions (risk: safe|sensitive, status: pending|approved|rejected|executed), audit_log (with prev_hash and hash fields), kb_articles. Implement REST endpoints for incident CRUD, log ingestion, alert webhook, action approve/reject, and analytics (MTTR, counts by severity/category). Add a state machine with validated transitions. Every state change and AI output must write to audit_log. Don't add auth. Write the code in small modules and add a seed script. Run it and fix errors before stopping.

Second prompt (Codex, AI pipeline)

In backend/pipeline, implement: drain3 log template parsing, rolling-z-score error-rate anomaly detection per service, correlation of logs/alerts within ±5 min using trace_id and a service dependency graph, TF-IDF similarity against historical incidents, and an LLM step that returns Pydantic-validated JSON {category, severity, priority, summary, probable_root_cause, confidence, evidence_log_ids, recommended_steps, proposed_actions}. If the LLM fails, return a rule-based fallback. Never let the LLM execute actions.

Frontend prompt (Antigravity)

Build a React + Tailwind + shadcn dashboard against this API [paste endpoints from Section 7]. Pages: Dashboard (severity cards, incident table with search/filter, live feed), Incident Detail (status stepper, evidence timeline, AI analysis panel with confidence and cited log lines, similar incidents, approve/reject buttons for sensitive actions, comment box), Knowledge Base, Analytics (Recharts), Audit Trail. Dark theme, clean, responsive.

10. Demo script (3 minutes)

1. Click **"Simulate Incident: bad deploy of checkout-service"** and the logs flood in.
2. The anomaly detector flags a 5xx spike and an incident is auto-created, classified as *Availability / SEV-1*.
3. Open the incident to see correlated logs across 3 services, the AI summary, and the probable root cause (DB connection pool exhausted after deploy v2.4.1) with 87% confidence and cited log lines.
4. Show the **similar past incident** and its fix.
5. The AI proposes *Rollback to v2.4.0* flagged **sensitive**. You click **Approve**.
6. The status moves to Mitigated, then Resolved. The AI drafts the postmortem.
7. Show the **audit trail**, then the **analytics page**.

11. Risks to avoid

- Starting the UI before the API contract is fixed.
- Spending an hour on real log source integrations.
- LLM output that isn't schema-validated.
- Not having a one-click failure simulator for the demo.
- Adding features in the last hour. Use it for testing and the README instead.

12. Suggested next step

Generate the **starter scaffold** first: the DB schema, the simulator, the LLM wrapper with fallback, and the `AGENTS.md` file. Then run the backend prompt (Section 9) in Codex and the frontend prompt in Antigravity in parallel.